



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo Práctico

19 de septiembre de 2026

Aprendizaje Automático I

Grupo 11

Integrante	LU	Correo electrónico
Bravo, Iván Agustín	001/01	email1@dominio.com
Kiss, María Delfina	72/23	kissdelfina@gmail.com
Lützow Holm, Eric	003/01	email3@dominio.com
Mendoza, Juan Cruz	004/01	email4@dominio.com
Rostan, Marcos Andrés	005/01	email5@dominio.com



Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón Cero + Infinito)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Conmutador: (+54 11) 5285-9721 / 5285-7400

<https://dc.uba.ar>

1. Separación de datos

Queremos partir el dataset de manera de tener una partición de held-out (conjunto de control o de evaluación) para evaluar nuestra configuración. Para eso, podemos tomar el estándar de usar el 15 % del total. Luego, es muy importante que esa partición sea representativa del dataset: que la proporción de etiquetas sea aleatoria y balanceada. Es decir, que la proporción de True y False sea la misma que en el resto del dataset. Además, ese 15 % no debe tener una propiedad que el resto del dataset no tenga y viceversa. Por ejemplo, que los valores de ciertos atributos sean muy diferentes al del resto.

Sabemos que del total, el 72 % de las instancias corresponden a 0 (False) y 28 % a 1 (True). La propuesta final entonces es mezclar primero las instancias y luego tomar esas proporciones para particionar el held-out. Una forma de hacer eso es utilizar los métodos `group_by` (agrupar por target) y luego `sample` (tomar una muestra al azar) de dataframe de Pandas. Queremos $0,15 \cdot 359$ de etiqueta 0 y $0,15 \cdot 141$ de etiqueta 1, o sea, 54 y 21 respectivamente.

2. Construcción de modelos

Buscamos construir y evaluar modelos de árboles de decisión, obteniendo una estimación realista de su performance. Separamos el desarrollo nuevamente de manera proporcional a sus etiquetas. Para eso, usamos `train_test_split` de `sklearn` estratificado. Nos quedamos con un 0,8 de entrenamiento y 0,2 de validación. Obtuvimos un accuracy sobre el subconjunto de validación de 0.6471.

Utilizamos `X_dev` e `y_dev` con un K-Fold estratificado, de manera que conserve la proporción de etiquetas.

Permutación	Acc (train)	Acc (val)	AUPRC (train)	AUPRC (val)	AUCROC (train)	AUCROC (val)
1	0.8265	0.6000	0.7153	0.3265	0.8626	0.5697
2	0.8265	0.7412	0.6862	0.4213	0.8037	0.6411
3	0.8353	0.6235	0.6981	0.2895	0.8506	0.4754
4	0.8265	0.6471	0.6900	0.3594	0.8429	0.6117
5	0.7971	0.6824	0.6491	0.3574	0.8538	0.5683
Promedios	0.8224	0.6588	0.6878	0.3508	0.8427	0.5732
Global	(NO)	0.6588	(NO)	0.3404	(NO)	0.5610

Cuadro 1: Resultados obtenidos para las cinco permutaciones y sus promedios.

Altura máxima	Criterio de corte	Accuracy (training)	Accuracy (validación)
3	Gini	0.8224	0.6588
5	Gini	0.9482	0.6447
Infinito	Gini	1.0000	0.6400
3	Entropía	0.8029	0.6541
5	Entropía	0.9253	0.6612
Infinito	Entropía	1.0000	0.5953

Cuadro 2: Resultados de Accuracy en entrenamiento y validación según hiperparámetros del árbol de decisión.

Del Cuadro 1, podemos ver que el modelo base (con altura máxima 3 y criterio Gini) logró un accuracy promedio de 0.8224 en entrenamiento y 0.6588 en validación. Además, al medir las predicciones globales sobre todo ese conjunto de validación, conseguimos un AUCROC de 0.5610 y un AUPRC de 0.3404.

En el Cuadro 2, podemos notar cómo a medida que aumentamos la altura máxima del árbol, mejor es la accuracy en los datos de entrenamiento de manera notoria, pero disminuye el accuracy

en los datos de validación. Esto es lo que conocemos como overfitting: el modelo aprende muy bien los datos que tiene de entrenamiento, pero después no es bueno para generalizar. Podemos ver que cuando el árbol no tiene restricción de altura no tiene errores en el conjunto de entrenamiento, pero después tiene una mala performance sobre los datos de validación.

De entre todas las opciones exploradas vemos que la que mayor accuracy de validación tiene es la de altura máxima 3 y criterio de corte Gini. Si bien los criterios de corte Gini y Entropía experimentan ambos overfitting al crecer la altura máxima, Gini tiene una ventaja en performance en los datos de validación por sobre Entropía.

3. Comparación de algoritmos

3.1. Árboles de decisión

Como hiperparámetros tuvimos en cuenta la profundidad máxima, la mínima cantidad de instancias que necesita un nodo para dividirse, la mínima cantidad de instancias de cada hoja, el criterio (Gini o Entropía) y la máxima cantidad de atributos, probando la raíz cuadrada y el logaritmo de la cantidad total.

Los mejores hiperparámetros encontrados fueron: `'min_samples_split': 10`, `'min_samples_leaf': 15`, `'max_features': 'log2'`, `'max_depth': 5`, `'criterion': 'gini'` Con un AUCROC score de 0.6740

3.2. KNN

Para KNN, primero hay que tener en cuenta que suele ser mejor escalar las features. Como pueden tener escalas muy distintas, las features de mayor rango pueden llegar a dominar la distancia. Hiperparámetros:

- **n_neighbours**: mientras un k chico se puede llegar a ajustar al ruido local de cada datapoint, un k grande simplemente se aproxima a predecir la clase mayoritaria.
- **weights**: permite que los puntos más cercanos pesen más.
- **metric**: que es la distancia que usamos para medir cercanía.

Mejores hiperparámetros encontrados para KNN: `'texttt'scaler': None`, `'knn_weights': 'distance'`, `'knn_n_neighbors': 21`, `'knn_metric': 'manhattan'` Mejor score AUCROC: 0.8474

3.3. SVM

Para SVM, también necesitamos escalar los features. Hiperparámetros:

- **C**: controla la regularización del SVM. Cuanto mayor es, obliga a SVM a tener un margen estricto, lo que puede llevar a overfitting, mientras que cuanto más chico es, tolera más puntos mal clasificados.
- **kernel**: nos dice si la frontera es lineal en el espacio de features, o se hace un mapping mediante el kernel trick para hacer un hiperplano en otro espacio.
- **degree**: para kernels polinómicos solamente debemos decidir hasta qué grado puede ser el polinomio.
- **gamma**: para el kernel RBF y poly, gamma define el radio de influencia de cada support vector.

Para **Gamma** y **C**, se suelen probar diferentes órdenes de magnitud. Para **degree**, grados mayores a 3 puede generar un espacio de features enorme y sobreajustar con pocas instancias.

Mejores hiperparámetros encontrados para SVM: `texttt'svm_kernel': 'poly', 'svm_gamma': 1, 'svm_degree': 2, 'svm_C': 1000, 'scaler': None` Mejor score AUCROC: 0.8587

Vemos que las top 5 mejores configuraciones tienen la misma AUCROC y sólo cambian el C y Gamma, indicando que sin escalar los mismos no tienen relevancia.

Índice	Params	Mean Test Score	Std Test Score
2	<code>{'svm_kernel': 'poly', 'svm_gamma': 1, ...}</code>	0.858743	0.055193
9	<code>{'svm_kernel': 'poly', 'svm_gamma': 0.0001, ...}</code>	0.858743	0.055193
23	<code>{'svm_kernel': 'poly', 'svm_gamma': 1, ...}</code>	0.858743	0.055193
29	<code>{'svm_kernel': 'poly', 'svm_gamma': 0.01, ...}</code>	0.858743	0.055193
25	<code>{'svm_kernel': 'poly', 'svm_gamma': 0.01, ...}</code>	0.858743	0.055193

Cuadro 3: Mejores configuraciones obtenidas mediante búsqueda de hiperparámetros.

Resulta muy llamativo ver que tanto para SVM y KNN, las mejores configuraciones de hiperparámetros según AUCROC fueron aquellas que no utilizaron el Scaler. Esto indica que las features con un rango más grande terminaron siendo aquellas que también tenían mayor poder predictivo.

3.4. LDA y Naive Bayes

Fold	LDA	Naive Bayes
1	0.7131	0.7425
2	0.6919	0.7582
3	0.6291	0.7589
4	0.6086	0.6926
5	0.5608	0.7357
Promedio	0.6407	0.7376

Cuadro 4: AUCROC por fold y promedio para LDA y Naive Bayes.

Vemos que el AUCROC de LDA es el peor con $AUC = 0.6407$. Esto puede deberse a que se está estimando una matriz de covarianza de 200x200 (por las 200 features con que se entrenan) con pocas instancias de train por fold, lo que resulta en una estimación ruidosa y casi singular. En este caso, podríamos utilizar el hiperparámetro de **shrinkage** que regulariza la matriz de covarianza.

Por otro lado, Naive Bayes, incluso sin haber hecho una búsqueda de hiperparámetros, tuvo un $AUC = 0.7376$, similar a la performance de SVM y KNN. Este modelo, al asumir independencia entre los features, sólo estima la media y varianza por cada gen, por lo que cada una de las estimaciones puede ser buena incluso con pocos datos. Uno de los hiperparámetros que podríamos modificar es **var_smoothing**, el cual añade una fracción de la varianza más grande de las features a cada una de las estimaciones de varianza. Esto funciona como una regularización de la varianza, y también ayuda a que si hay una feature con varianza casi cero, un valor distinto en test no reciba probabilidad casi cero, dominando el producto.

4. Evaluación de performance

5. Conclusiones